Assignment 1

Fabian Blasch, Nik Gabrovsek, Patrick Gyasi, Gregor Steiner

26.05.2021

Time Series Analysis

1

We can use the **quantmod** package to download the data from the FRED database.

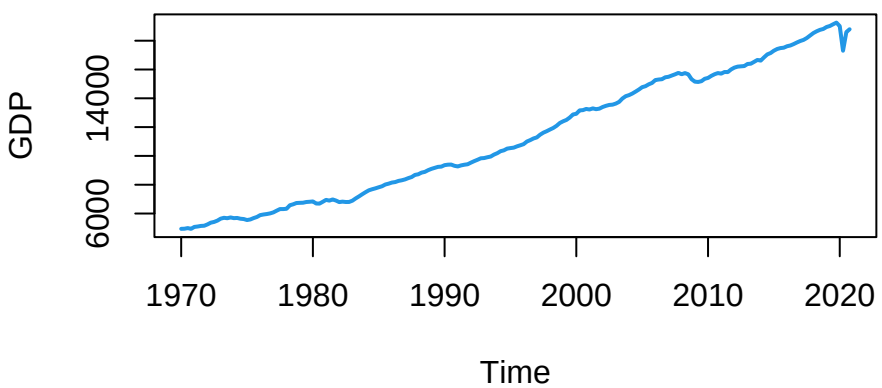
```
# Download data
quantmod::getSymbols("GDPC1", src = "FRED")
```

```
## [1] "GDPC1"
```

```
gdp.ts <- ts(GDPC1, start = c(1947, 1), frequency = 4)

# Filter
gdp.ts <- window(gdp.ts, start = c(1970, 1), end = c(2020, 4))
```

After filtering for the time period we are interested in, we can now plot the series.

```
# Plot
plot(gdp.ts, lwd = 2, col = 4, ylab = "GDP")
```



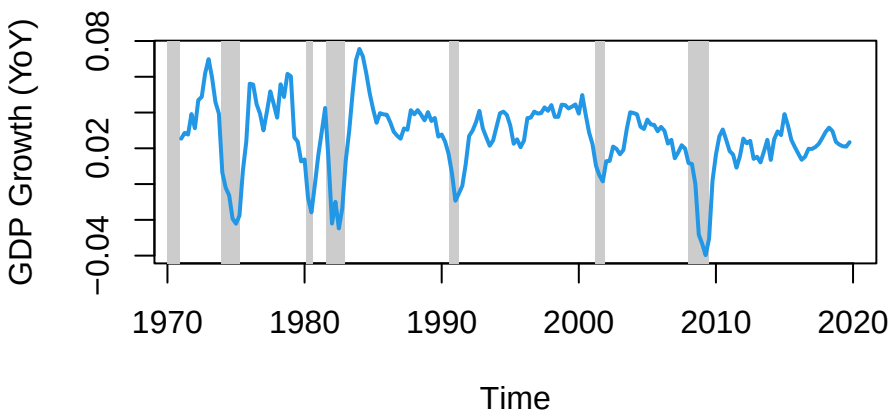
2

Trend is a slow change that occurs over a long period of time. In our case trend is clearly positive. Cyclical component is movement around our trend line, its of short time nature.The cycle component is closely related to business cycles. Time series with trend in it are normally not stationary. Stationary time series have no

predictable patterns in long term which is clearly not the case for GDP. Main idea of detrending is to get rid of a trend in a time series. Detrending changes our un-stationary time series into a stationary one. We know different ways of detrending. First one I will mention is usage of growth rates, we basically take a difference of logarithm between two data points. This gives us a growth rate time series which is stationary. Next one I will mention is linear/nonlinear detrending, if we assume our trend is linear, we can estimate linear regression of our time-series this will give us a trend and we can estimate our detrended time-series with observation of deviations from linear regression line. Lastly we can use filters to detrend data, there are various different filter such as HP and Bandpass filter. They are similar to linear detrending, but we can have smoother line of best-fit as trend line is not necessarily linear.

3

```r
# Compute y/y growth rates
gdp.growth <- diff(gdp.ts, 4) / lag(gdp.ts, 4)

# Plot
plot(gdp.growth, type = "n", ylab = "GDP Growth (YoY)")
tis::nberShade()
lines(gdp.growth, lwd = 2, col = 4)
```



4

```r
# Test for stationarity
tseries::adf.test(gdp.growth)
```

```
## Warning in tseries::adf.test(gdp.growth): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  gdp.growth
## Dickey-Fuller = -4.5746, Lag order = 5, p-value = 0.01
## alternative hypothesis: stationary
```

We do reject the null-hypothesis of non-stationarity. Thus, the series is stationary.

5

We want to estimate the model

$$y_t = c + \varphi_1 y_{t-1} + \varphi_2 y_{t-2} + \varepsilon_t$$

in a Bayesian fashion. In matrix notation this reads as

$$y = X\varphi + \varepsilon,$$

where $y := [y_3, ..., y_T]', \varphi := [c, \varphi_1, \varphi_2]', \varepsilon := [\varepsilon_3, ..., \varepsilon_T]'$ and

$$X := \begin{bmatrix} 1 & y_2 & y_1 \\ \vdots & \vdots & \vdots \\ 1 & y_{T-1} & y_{T-2} \end{bmatrix}.$$

We obviously lose the first two observations, leaving us with $T-2$ observations.

The first step is choosing prior distributions for c, φ_1, φ_2. A simple normal prior might be a good start. Thus,

$$\varphi \sim N_3(\mu_0, \Sigma_0),$$

where $\mu_0 = [\mu_c, \mu_{\varphi_1}, \mu_{\varphi_2}]'$ and $\Sigma_0 = diag(\sigma_c^2, \sigma_{\varphi_1}^2, \sigma_{\varphi_2}^2)$.

Then, we only need a prior distribution for the error variance, where we will use an Inverse Gamma Distribution as in the linear model:

$$\sigma^2 \sim IG(c_0, d_0).$$

And we know that

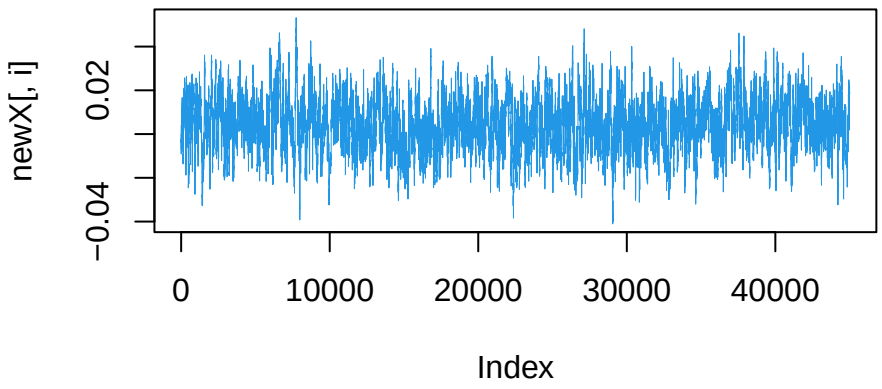
$$y|\varphi, \sigma^2 \sim N_{T-2}(X\varphi, \sigma^2 I_{T-2}).$$

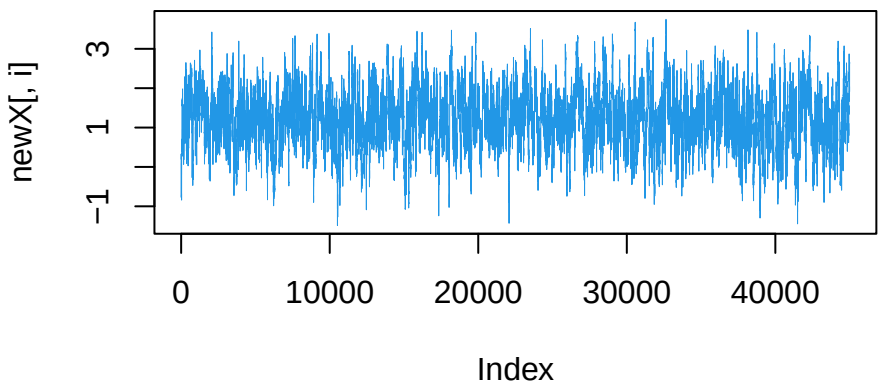
Now, we can start setting up the Metropolis algorithm to estimate the posterior density.

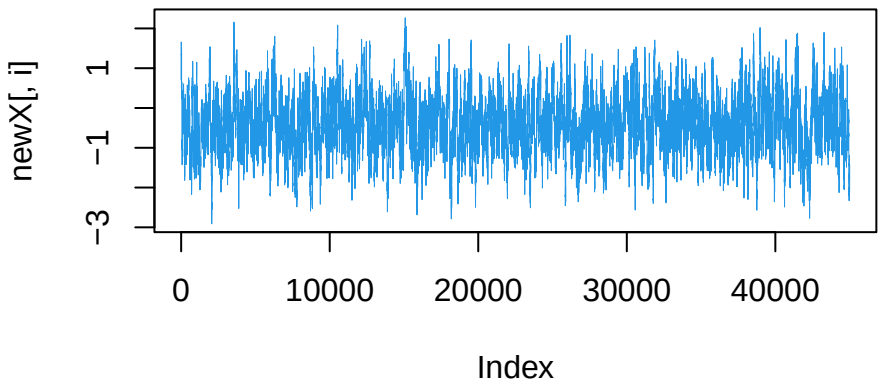
```r
# Set up dependant variable
# time period
t <- length(gdp.growth)


# lags
interc <- rep(1, t - 2)

# we remove the first two obs since the L(2) var start in t = 2
y1 <- gdp.growth[-(1:2)]

# lagged vars
y_l1 <- gdp.growth[-c(1, t)]
y_l2 <- gdp.growth[-c(t - 1, t)]

# bind to a matrix for regression
X <- cbind(interc, y_l1, y_l2)

# Metropolis - Hastings algorithm #

# Normal Prior on c
mu0 <- 0
sig0 <- sqrt(100)
```

```r
# IG prior on on the variance
s0 <- 2
r0 <- 1

# normal prior on phi
pmu0 <- 0
sig0 <- sqrt(100)

# Proposal variances
eta2.c <- 0.005
eta2.phi <- 0.4

# storage and starting values
sig2.draw <- 1
phi.draw <- c(0, 0, 0)
accept <- 0
sig0 <- sqrt(100)

# Function wrapper
MCM <- function(nsave = 45000, nburn = 10000, covmat = X, y = y1){
  # storage
  phi.store  <- matrix(NA, nsave, 3)
  sig2.store <- matrix(NA, nsave, 1)
  ntot  <- nsave + nburn

  # loop
  for(i in 1:ntot){
    # create a proposal for phi
    phi.prop <- mapply(function(x, y) rnorm(1, x, y), phi.draw, c(eta2.c, eta2.phi, eta2.phi))

    # evaluate the uniform prior for current draw and proposal
    prior.current <- sum(dnorm(phi.draw[1], mu0, sig0, log = T),
                         dnorm(phi.draw[2:3], pmu0, sig0, log = T))

    prior.prop    <- sum(dnorm(phi.prop[1], mu0, sig0, log = T),
                         dnorm(phi.prop[2:3], pmu0, sig0, log = T))

    # evaluate the normal likelihood for current draw and proposal
    ll.current    <- sum(dnorm(y, X %*% phi.draw, sqrt(sig2.draw), log = T))
    ll.prop       <- sum(dnorm(y, X %*% phi.prop, sqrt(sig2.draw), log = T))

    # log likelihood ratio and difference
    c.p.current   <- ll.current + prior.current
    c.p.prop      <- ll.prop    + prior.prop

    # finally, we can compute the likelihood ratio
    ll.ratio      <- c.p.prop - c.p.current

    # Discard if ll smaller than draw
    if(ll.ratio > log(runif(1))){ # if our proposal is likely "enough"

      #replace the current draw for phi with our proposal
      phi.draw <- phi.prop
```

```r
      #add 1 to our acceptance counter
      accept   <- accept + 1
    }

     # STEP 2: SAMPLE SIGMA2 GIVEN phi & DATA

    # compute errors
    e  <- y - X %*% phi.draw
    # sum of squared residuals (equivalent to sum((e)^2))
    ee <- t(e) %*% e

    # compute posterior quantities
    s <- s0 +  t/2
    r <- r0 + ee/2

    # sample sigma2 from inverse gamma posterior
    sig2.draw <- 1 / rgamma(1, s, r)

  # STEP 3: STORE POSTERIOR SAMPLES

    #only after burn-in period!

    if(i > nburn){

    phi.store[i - nburn,]  <- phi.draw
    sig2.store[i - nburn,] <- sig2.draw

    }
  }

  return(list(phi.store,
              accept / i))
}

# run
mcm1 <- MCM()

# trace
apply(mcm1[[1]], 2, plot, type = "l", col = 4, lwd = 0.01)
```


newX[, i]
0.02
−0.04
0
10000
20000
30000
40000
Index


newX[, i]
3
1
−1
0
10000
20000
30000
40000
Index

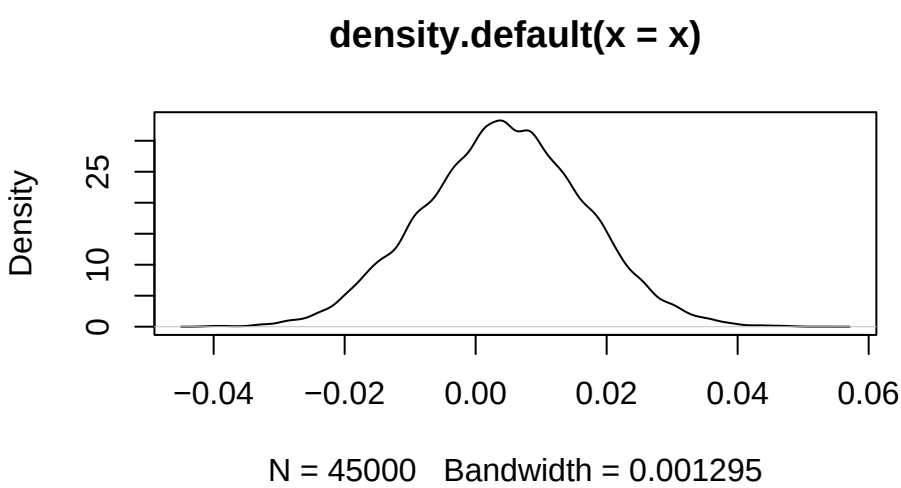


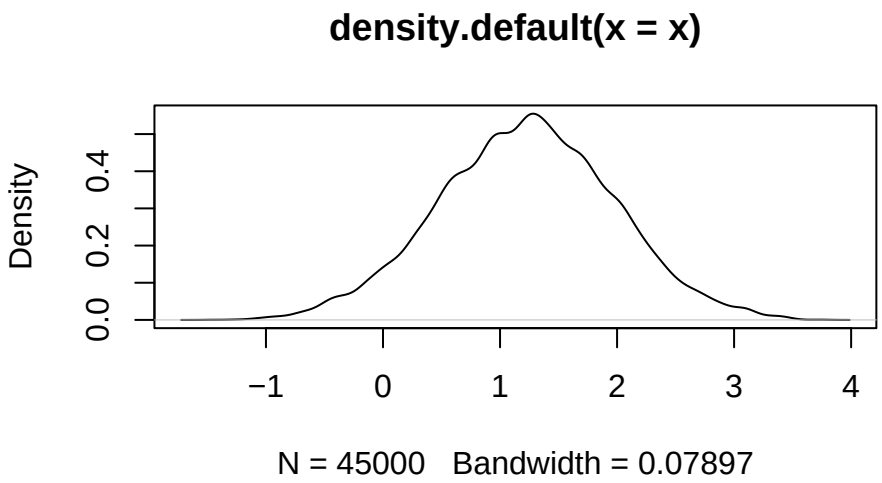

```
## NULL
```

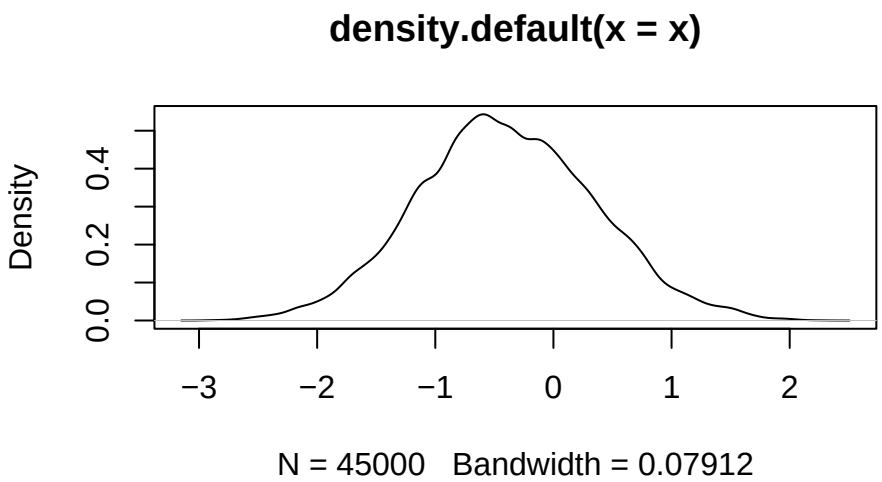
```
# accept
mcm1[[2]]
```

```
## [1] 0.3735636
```

```
# dis
apply(mcm1[[1]], 2, function(x){
  plot(density(x))
})
```








```
## NULL
```

7

To simulate the IRF for our AR(2) function, we can make use of the fact that any AR(p) process can be written as a vector AR(1) process. Once brought into the so called companion form we can easily obtain the IRF from the j'th power of the (1,1) element of the companion matrix.

```
# first exclude all non stable draws, i.e, |phi| > 1
bool <- apply(mcm1[[1]], 1, function(x) sum(abs(x)) < 1)



# subset
phi.clean <- mcm1[[1]][bool, ]

# median  and quantiles of estimates
```

```r
est.med <- apply(phi.clean, 2, function(x){
  c(median(x), quantile(x, probs = c(0.7, 0.3)))
})



# Companion Matrix
phi.comp <- lapply(as.data.frame(t(est.med[, 2:3])), function(x){
  as.matrix(rbind(x, c(1, 0)))
})
est.med
```

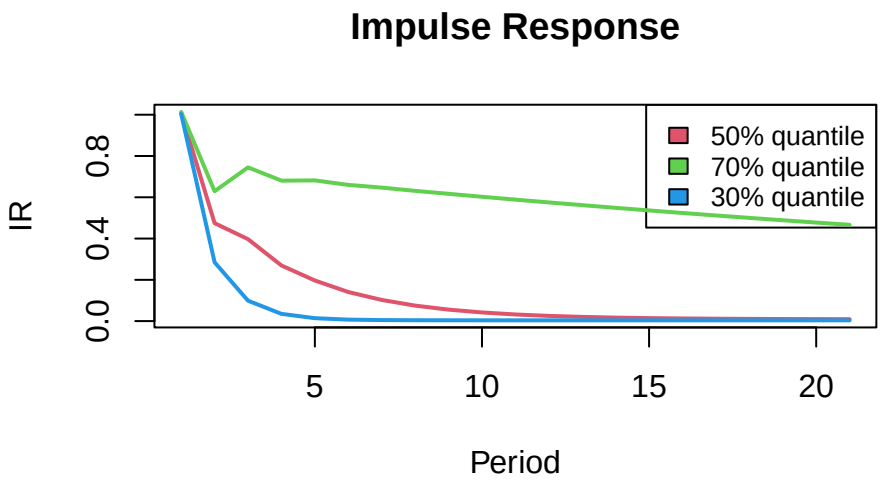
```
##            [,1]      [,2]       [,3]
##     0.008520106 0.4659569 0.17142466
## 70% 0.013559662 0.6155807 0.35231130
## 30% 0.003813077 0.2810199 0.01608933
```

```r
# number of periods and storage for impulse response
j <- 20
Imp.store <- matrix(NA, nrow = j + 1, ncol = 3)

# Now we can pick out the (1, 1) element of phi.comp^j to recover the impulse response
# at horizon j to a unit shock at time t. We can easily do this within a loop.
# We can use library(expm) to obtain a matrix power function.

library(expm)
```

```
## Lade nötiges Paket: Matrix

##
## Attache Paket: 'expm'

## Das folgende Objekt ist maskiert 'package:Matrix':
##
##     expm
```

```r
col <-  2:4

invisible(
Map(function(x, y, z){
  # storage
  st <- rep(NA, j + 1)

  # loop
  for(l in 0:20){
     st[l + 1] <- (x %^% l)[1, 1] + y
  }

  # plot
  if(x[1,1] == phi.comp[[1]][1, 1]){
    plot(st, type = "l", col = z, main = "Impulse Response", lwd = 2, ylab = "IR",
         xlab = "Period")
    legend("topright", fill = col, legend = c("50% quantile", "70% quantile", "30% quantile"),
           cex = 0.8)
  } else {
    lines(st, col = z, lwd = 2)
```

```
  }

}, phi.comp, est.med[, 1], col))
```



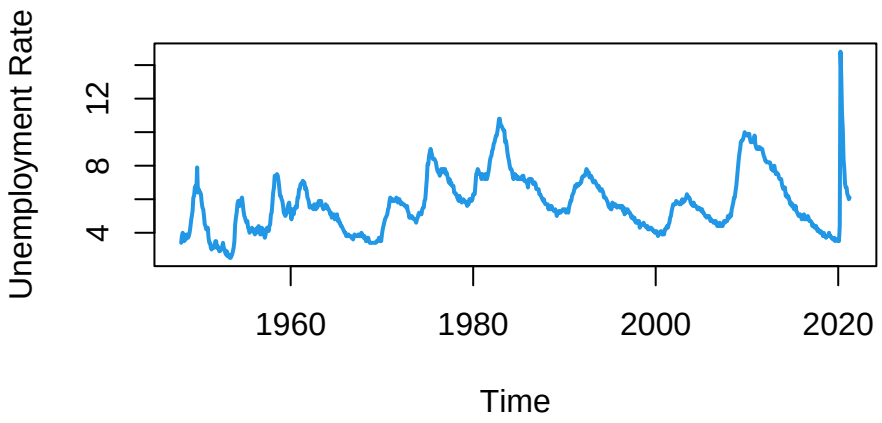
Prior Sensitivity

Again, we can use the **quantmod** package to download the data from the FRED database.

```
# Download data
quantmod::getSymbols("UNRATE", src = "FRED")
```

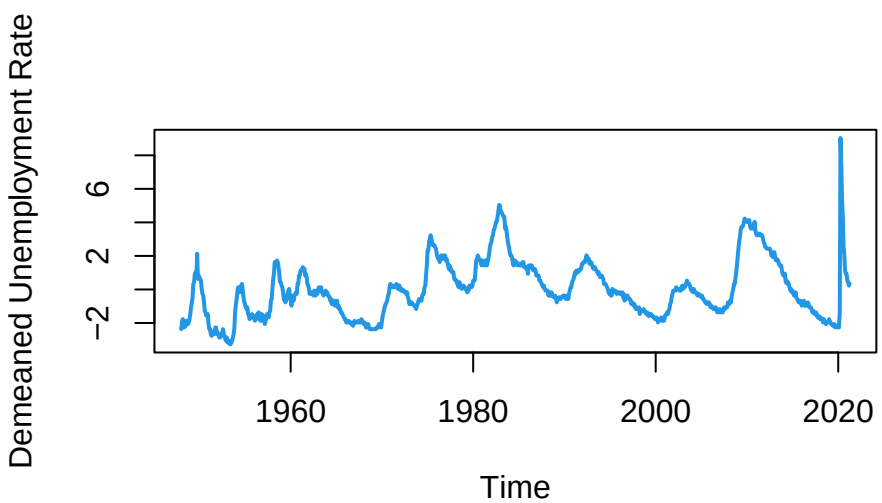
```
## [1] "UNRATE"
```

```
un.ts <- ts(UNRATE, start = c(1948, 1), frequency = 12)

# Plot
plot(un.ts, col = 4, lwd = 2, ylab = "Unemployment Rate")
```



1

```
# Demean series
un.demeaned <- un.ts - mean(un.ts)

# Plot
plot(un.demeaned, col = 4, lwd = 2, ylab = "Demeaned Unemployment Rate")
```



2

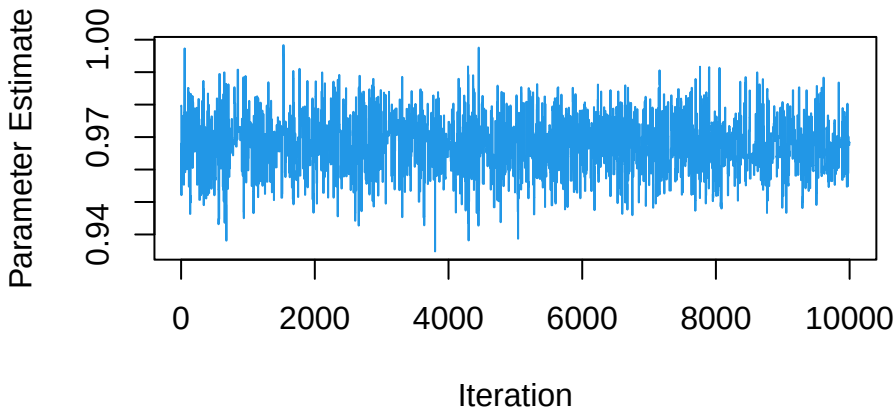
We will use the Metropolis algorithm used in the lecture.

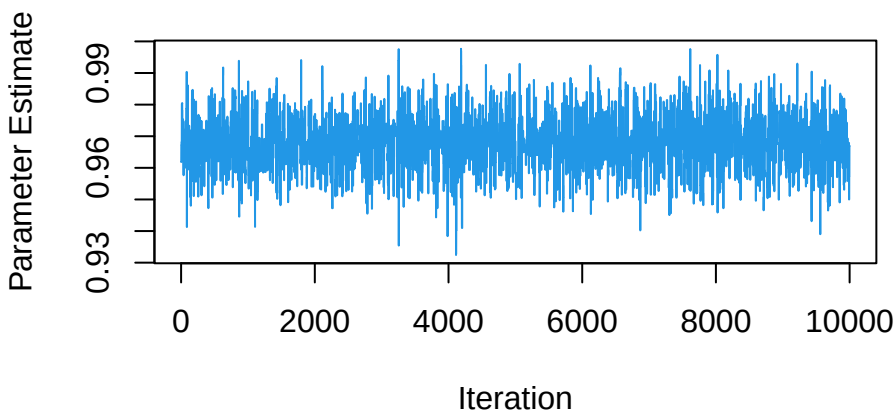
```
metro.est <- function(y, x, prior, prior.params){


  # IG prior on sigma2
```

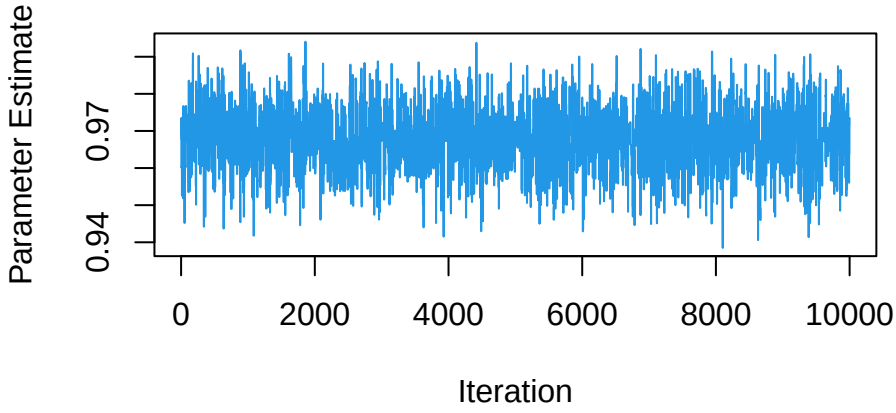
```r
c0 <- 2
d0 <- 1

# Algorithm Setup
nburn <- 1e4
nsave <- 1e4
ntot <- nburn + nsave
t <- length(x)
eta2  <- 0.0161

sig2.draw <- 1
phi.draw  <- 0.5
accept    <- 0


# Storage matrix
phi.store <- matrix(NA, nsave, 1)

sig.store <- matrix(NA, nsave, 1)


# Metropolis algorithm loop
for (i in 1:ntot) {

  ## STEP 1:

  # Create proposal
  phi.prop <- rnorm(1, phi.draw, sqrt(eta2))


  # Evaluate prior for proposal
  prior.current <- do.call(paste0("d", prior), list(phi.draw, prior.params[1], prior.params[2], log =
  prior.prop    <- do.call(paste0("d", prior), list(phi.prop, prior.params[1], prior.params[2], log =

  # Evaluate LL
  ll.current <- sum(dnorm(y, x * phi.draw, sqrt(sig2.draw), log = TRUE))
  ll.prop <- sum(dnorm(y, x * phi.prop, sqrt(sig2.draw), log = TRUE))

  # And add prior
  c.p.current <- ll.current + prior.current
  c.p.prop <- ll.prop + prior.prop

  # And LR
  ll.ratio <- c.p.prop - c.p.current

  if(ll.ratio > log(runif(1))){ #if our proposal is likely "enough"

      #replace the current draw for rho with our proposal
      phi.draw <- phi.prop

      #add 1 to our acceptance counter
      accept   <- accept + 1
```

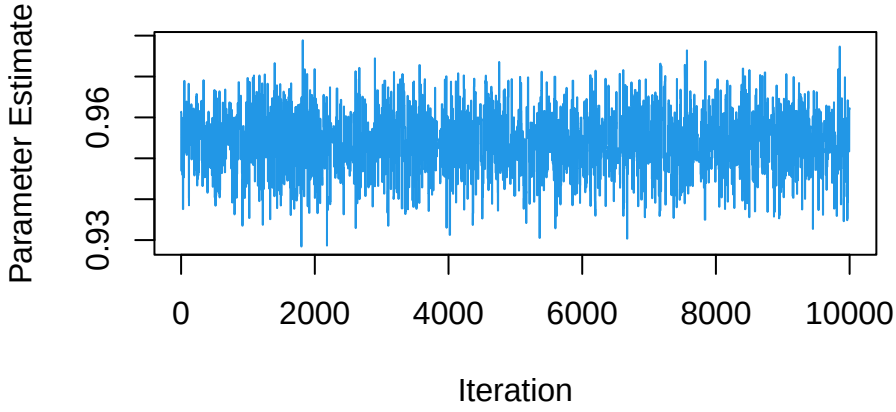
```r
      }

    # Tune proposal variance during first 50% of burn-in
    if (i < 0.5 * nburn) {
      if(accept / i > 0.3) eta2 <- eta2 * 1.01
      if(accept / i < 0.15) eta2 <- eta2 * 0.99
    }


    ## STEP 2: SAMPLE SIGMA2 GIVEN PHI & DATA

    # compute errors
    e  <- y - x * phi.draw

    # sum of squared residuals (equivalent to sum((e)^2))
    ee <- t(e) %*% e

    # compute posterior quantities
    ck <- c0 +  t/2
    dk <- d0 + ee/2
    # sample sigma2 from inverse gamma posterior
    sig2.draw <- 1 / rgamma(1, ck, dk)

    # STEP 3: STORE POSTERIOR SAMPLES

    #only after burn-in period!

    phi.store[i - nburn,]  <- phi.draw
    sig.store[i - nburn,] <- sig2.draw



  }

  return(list(Phi = phi.store, Sig2 = sig.store))

}

# Estimate posteriors
yy <- un.demeaned[-1]
xx <- un.demeaned[-length(un.demeaned)]

priors <- c("norm", "norm", "beta", "beta", "beta")
prior.params <- list(c(0,1), c(1, 1), c(1, 1), c(10, 10), c(0.1, 0.1))

res <- list()

for (i in 1:length(priors)) {
  res[[i]] <- metro.est(yy, xx, priors[i], prior.params[[i]])
}


# Traceplots and densities
```

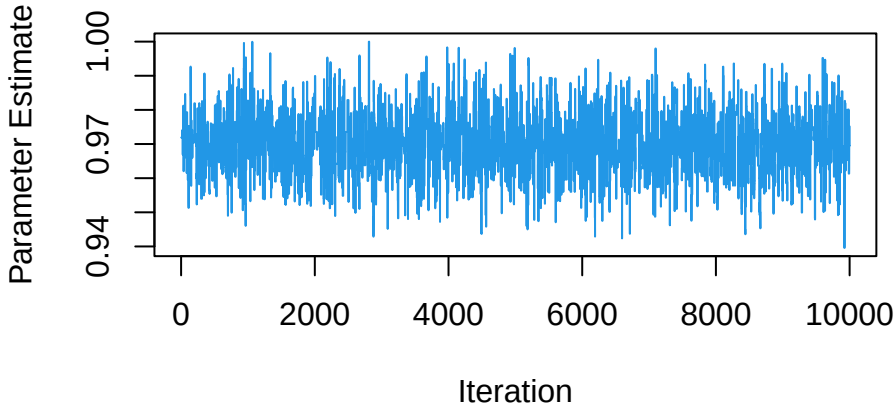
```
invisible(lapply(res, function(x) plot.ts(x[[1]], xlab = "Iteration", ylab = "Parameter Estimate", col =
```



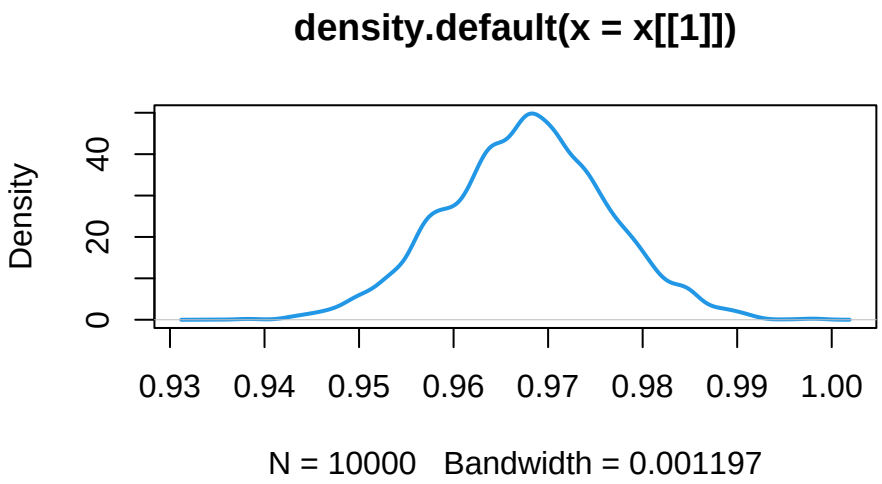


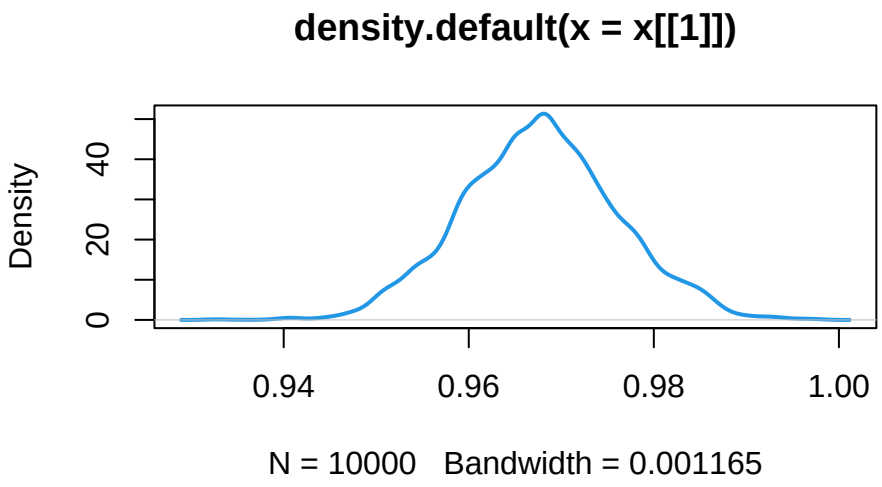

Parameter Estimate
0.94
0.97
0
2000
4000
6000
8000
10000
Iteration

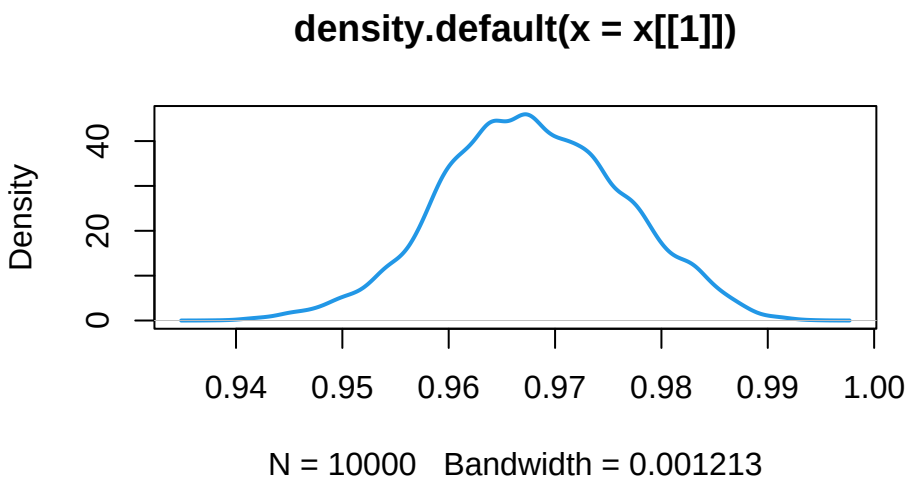

Parameter Estimate
0.93
0.96
0
2000
4000
6000
8000
10000
Iteration

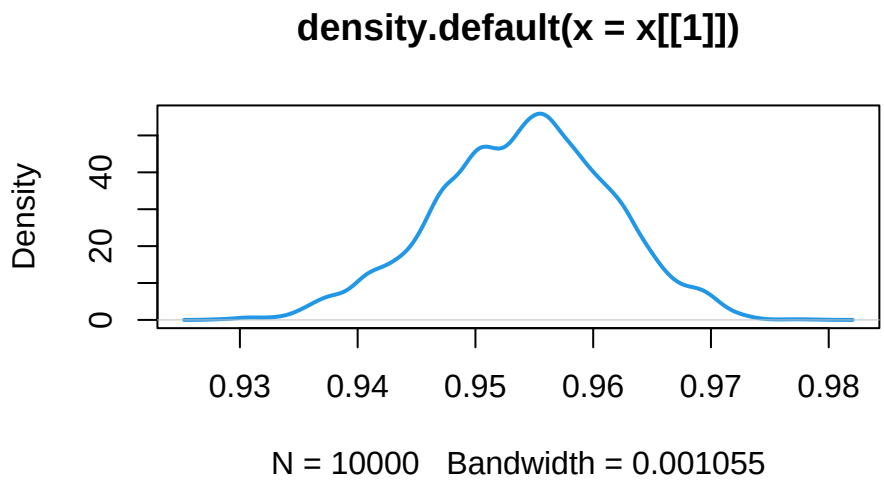


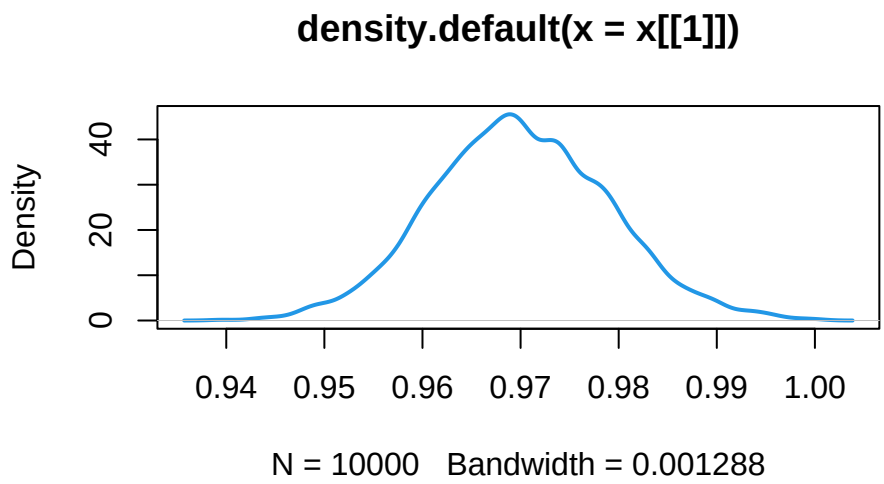

```
invisible(lapply(res, function(x) plot(density(x[[1]]), lwd = 2, col = 4)))
```




density.default(x = x[[1]])
Density
0
20
40
0.94
0.96
0.98
1.00
N = 10000 Bandwidth = 0.001165

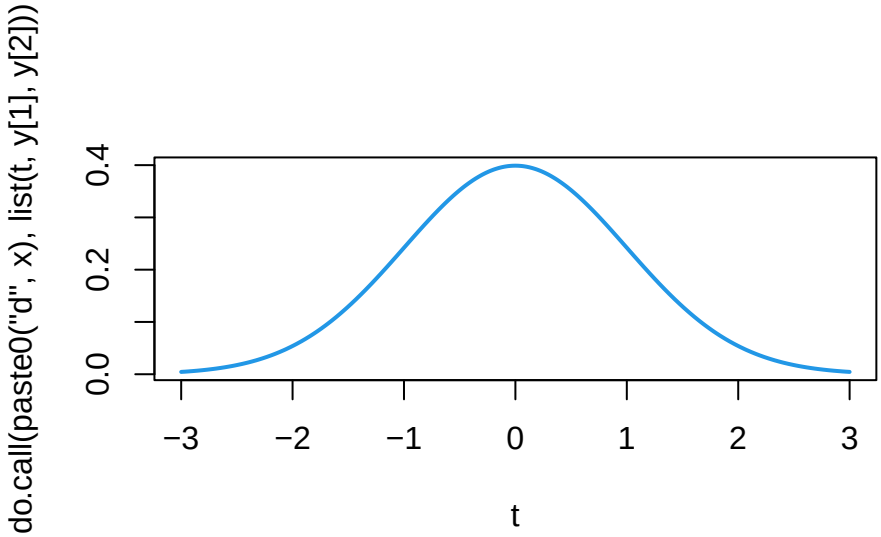

density.default(x = x[[1]])
Density
0
20
40
0.94
0.95
0.96
0.97
0.98
0.99
1.00
N = 10000 Bandwidth = 0.001213

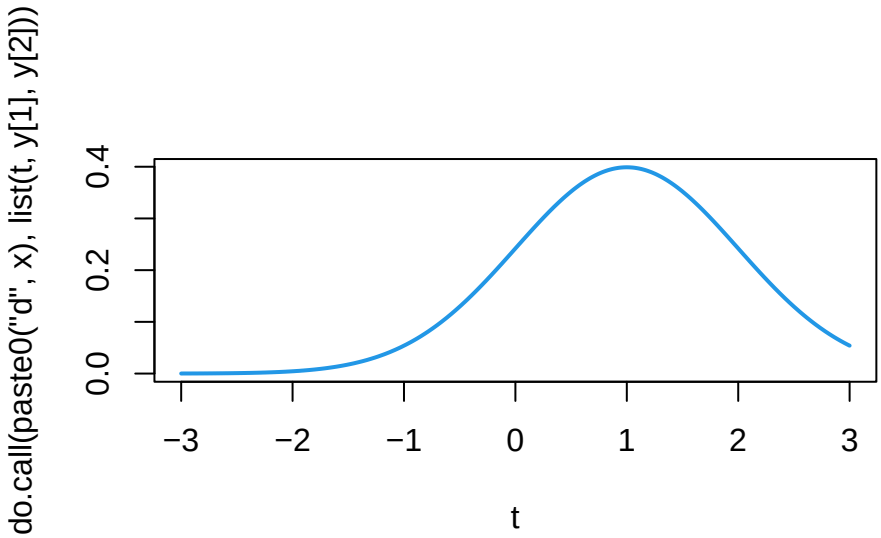


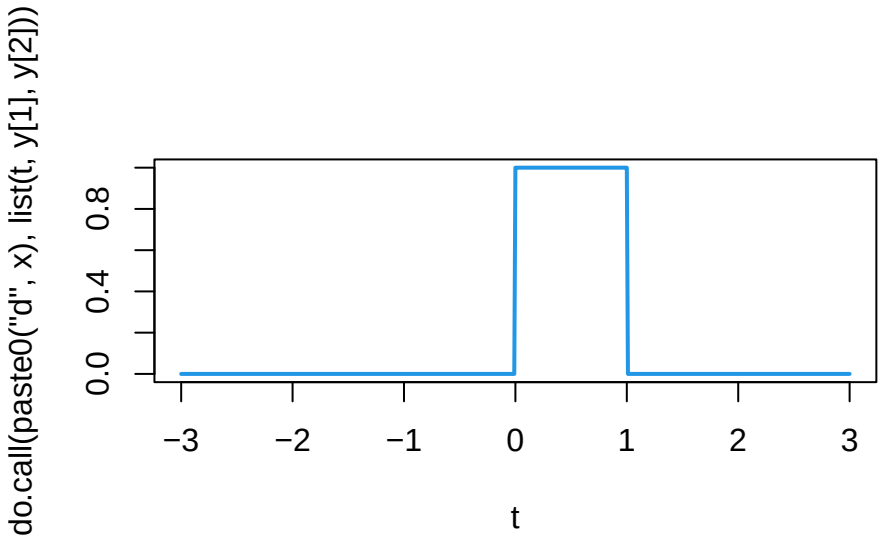


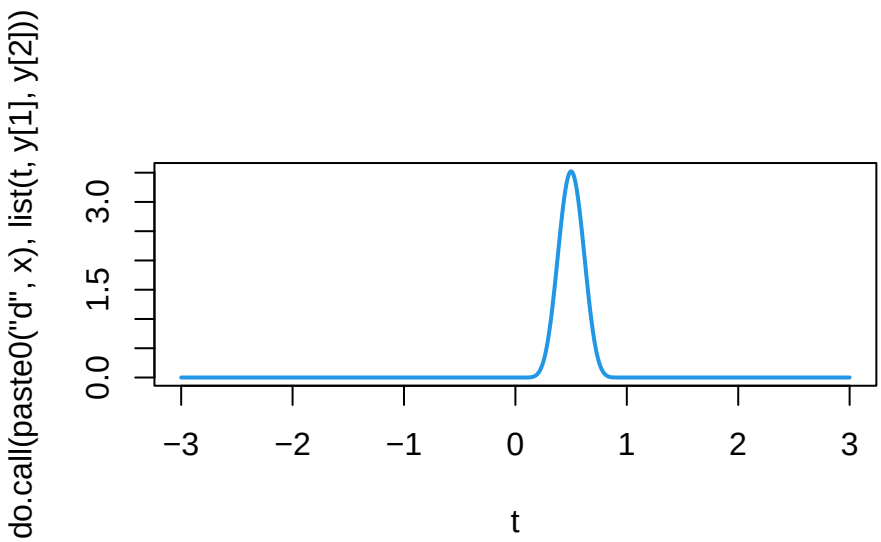
3

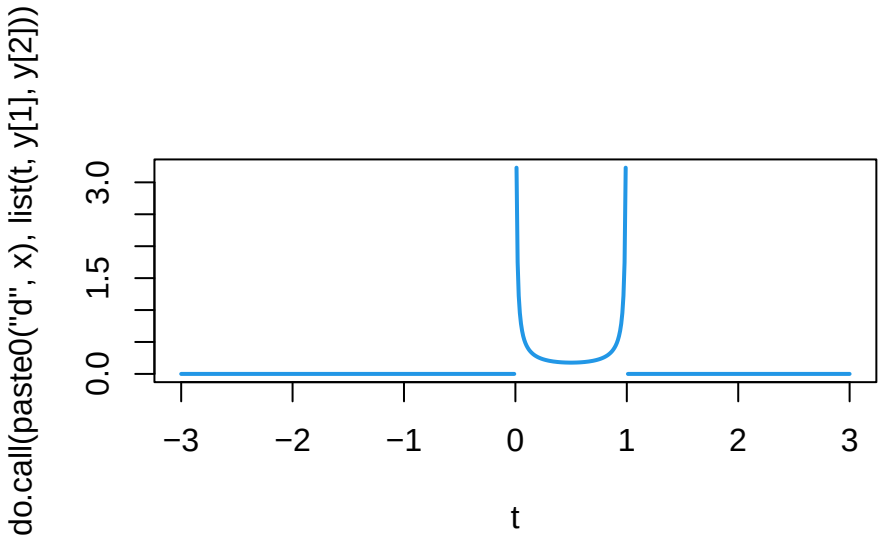
```
# Plot prior densities
invisible(Map(function(x, y){
  t <- seq(-3, 3, 0.01)
  plot(t, do.call(paste0("d", x), list(t, y[1], y[2])),
       type = "l", lwd = 2, col = 4)
}, priors, prior.params))
```


do.call(paste0("d", x), list(t, y[1], y[2]))
0.0
0.2
0.4
−3
−2
−1
0
1
2
3
t


do.call(paste0("d", x), list(t, y[1], y[2]))
0.0
0.2
0.4
−3
−2
−1
0
1
2
3
t


do.call(paste0("d", x), list(t, y[1], y[2]))
0.0
0.4
0.8
−3
−2
−1
0
1
2
3
t


do.call(paste0("d", x), list(t, y[1], y[2]))
0.0
1.5
3.0
−3
−2
−1
0
1
2
3
t


do.call(paste0("d", x), list(t, y[1], y[2]))
0.0
1.5
3.0
−3
−2
−1
0
1
2
3
t